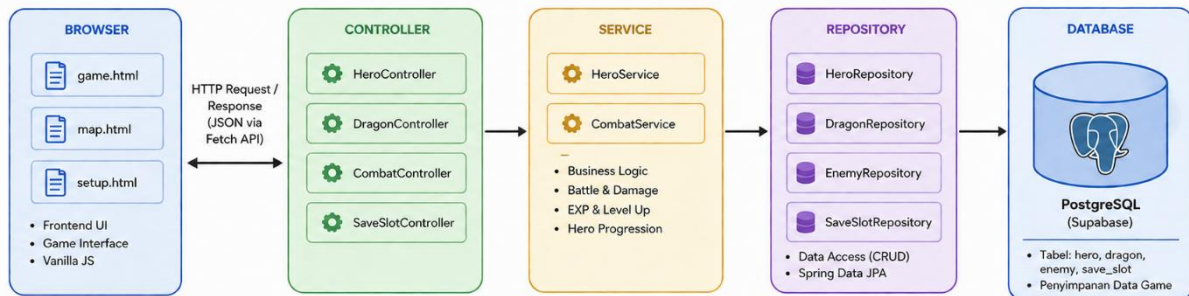


Dragon Monster Game

Architecture Diagram



Penjelasan Alur

Pada project Dragon Monster Game, alur komunikasi dimulai dari Browser yang menjalankan antarmuka permainan berbasis HTML, CSS, dan JavaScript. Browser mengirimkan request menggunakan Fetch API dengan format JSON ke backend yang dibangun menggunakan Spring Boot.

Request pertama kali diterima oleh Controller, yang bertugas memvalidasi data dan menentukan layanan (Service) yang akan dipanggil. Selanjutnya, Service Layer menjalankan seluruh logika bisnis permainan, seperti sistem pertarungan (combat), perhitungan damage, pemberian experience (EXP), proses level up, serta pengelolaan data hero dan musuh.

Apabila Service membutuhkan penyimpanan atau pengambilan data, maka Service akan memanggil Repository Layer. Repository menggunakan Spring Data JPA untuk melakukan operasi CRUD tanpa perlu menuliskan query SQL secara manual. Repository kemudian berinteraksi dengan PostgreSQL Database yang dihosting pada Supabase melalui Hibernate ORM.

Setelah proses selesai, hasil pengolahan data dikembalikan secara berurutan dari Repository → Service → Controller, kemudian Controller mengirimkan JSON Response kembali ke Browser sehingga tampilan permainan dapat diperbarui secara real-time.

Tabel Layer

Layer	Kelas / Interface	Peran
Controller	HeroController, DragonController, CombatController, SaveSlotController	Menerima HTTP Request dari browser, memvalidasi data, memanggil Service, dan mengembalikan Response JSON.
Service	HeroService, CombatService	Menjalankan logika bisnis permainan seperti sistem pertarungan, perhitungan damage, EXP, level up, serta pengelolaan hero dan musuh.
Repository	HeroRepository, DragonRepository, EnemyRepository, SaveSlotRepository	Mengakses database menggunakan Spring Data JPA dan menyediakan operasi CRUD terhadap seluruh data game.
Entity	Hero, Dragon, Enemy, SaveSlot, ElementType	Merepresentasikan struktur tabel database menggunakan anotasi @Entity pada Hibernate/JPA.
DTO	AttackRequest, AttackResponse	Digunakan sebagai objek pertukaran data (request dan response) antara frontend dan backend selama proses pertarungan.

Database	PostgreSQL (Supabase)	Menyimpan seluruh data permainan seperti data hero, dragon, enemy, save slot, dan progres permainan.
-----------------	-----------------------	--

Alasan Pemilihan Arsitektur

Arsitektur Layered Architecture (Controller – Service – Repository – Database) dipilih karena merupakan pola arsitektur yang direkomendasikan pada framework Spring Boot. Pemisahan setiap layer membuat tanggung jawab masing-masing komponen menjadi lebih jelas sehingga kode lebih mudah dikembangkan, diuji, dan dipelihara.

Pada project Dragon Monster Game, seluruh logika permainan ditempatkan pada Service Layer, sedangkan Controller hanya bertugas menerima request dan mengembalikan response. Pendekatan ini membuat perubahan terhadap mekanisme permainan, seperti penambahan fitur combat, level up, atau sistem inventory, dapat dilakukan tanpa mengubah struktur Controller.

Selain itu, penggunaan Spring Data JPA pada Repository mempermudah proses akses database karena sebagian besar operasi CRUD dapat dilakukan secara otomatis tanpa menuliskan query SQL secara manual. Hal ini mempercepat proses pengembangan sekaligus mengurangi kemungkinan kesalahan pada pengolahan data.

Frontend dan backend juga dijalankan dalam satu aplikasi Spring Boot, sehingga proses deployment menjadi lebih sederhana karena hanya memerlukan satu server aplikasi. Seluruh komunikasi antara frontend dan backend menggunakan REST API dengan format JSON, sedangkan penyimpanan data menggunakan PostgreSQL yang

dihosting pada Supabase sehingga data permainan dapat disimpan secara terpusat dan mudah dikelola.